

1. WHAT IS A PRIMARY KEY?

A Primary Key is a column (or a set of columns) in a database table that uniquely identifies each row/record in that table.

KEY CHARACTERISTICS:

Characteristic	Description
Unique	No two rows can have the same primary key value
Not NULL	Every row must have a value; it cannot be empty
Immutable	The value should not change over time
Single per table	A table can have only ONE primary key
Indexed	Automatically creates an index

	for fast searching
--	-----------------------

EXAMPLE:

sql

```
CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,    -- Primary key - uniquely identifies each
    employee
    emp_name VARCHAR(100),
    emp_email VARCHAR(100)
);

-- Valid inserts
INSERT INTO Employee VALUES (1, 'John Doe', 'john@email.com');
INSERT INTO Employee VALUES (2, 'Jane Smith', 'jane@email.com');

-- Invalid insert (duplicate emp_id)
INSERT INTO Employee VALUES (1, 'Bob Brown', 'bob@email.com'); --
ERROR!
```

COMPOSITE PRIMARY KEY:

A primary key can consist of multiple columns:

sql

```
CREATE TABLE Enrollment (
    student_id INT,
    course_id INT,
    enrollment_date DATE,
    PRIMARY KEY (student_id, course_id)    -- Both columns together
    uniquely identify a row
);
```

2. WHAT IS A FOREIGN KEY AND WHY IS IT NEEDED?

DEFINITION:

A Foreign Key is a column (or set of columns) in one table that references the Primary Key of another table. It establishes a link/relationship between two tables.

WHY IS IT NEEDED?

Foreign keys are essential for maintaining Referential Integrity in a relational database. Here are the key reasons:

1. MAINTAINS DATA CONSISTENCY

Prevents orphan records by ensuring that every foreign key value exists in the referenced table.

sql

```
CREATE TABLE Department (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);  
  
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(100),  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)  
);  
  
-- Valid: dept_id 1 exists  
INSERT INTO Employee VALUES (1, 'John', 1); -- OK  
  
-- Invalid: dept_id 99 doesn't exist  
INSERT INTO Employee VALUES (2, 'Jane', 99); -- ERROR! Foreign key  
constraint fails
```

2. PREVENTS DATA DELETION THAT WOULD CREATE ORPHANS

Restricts deletion of parent records that are still referenced by child records.

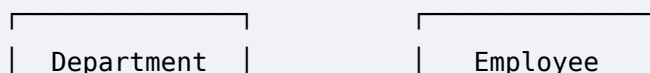
sql

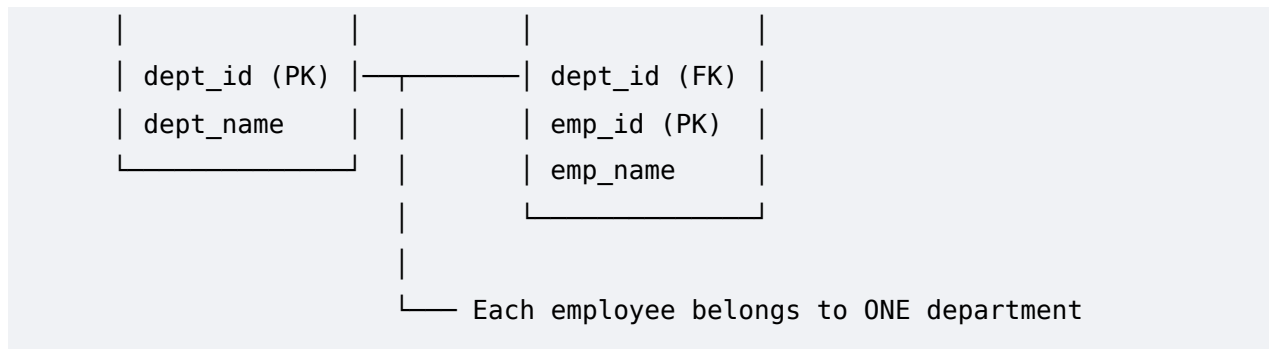
```
-- If employee John (dept_id=1) exists, this will fail  
DELETE FROM Department WHERE dept_id = 1; -- ERROR! Referenced by  
Employee
```

3. ENFORCES RELATIONSHIPS BETWEEN TABLES

Defines how tables relate to each other (one-to-many, many-to-one, etc.)

text





4. SUPPORTS CASCADING OPERATIONS

Can automatically update or delete related records:

sql

```
-- ON DELETE CASCADE: Deletes employees when their department is deleted
FOREIGN KEY (dept_id) REFERENCES Department(dept_id) ON DELETE CASCADE

-- ON UPDATE CASCADE: Updates employee dept_id when department dept_id
changes
FOREIGN KEY (dept_id) REFERENCES Department(dept_id) ON UPDATE CASCADE
```

5. ENABLES JOIN OPERATIONS

Foreign keys allow tables to be joined meaningfully:

sql

```
-- Join Employee and Department using Foreign Key
SELECT e.emp_name, d.dept_name
FROM Employee e
JOIN Department d ON e.dept_id = d.dept_id;
```

SUMMARY COMPARISON

Feature	Primary Key	Foreign Key
Purpose	Uniquely identify a row	Link tables and maintain referential integrity

Uniqueness	Must be unique	Can have duplicate values
NULL values	Cannot be NULL	Can be NULL (unless specified)
Number per table	Only ONE	Can have multiple
References	Does not reference another table	References the Primary Key of another table
Creates index?	Yes, automatically	Not automatically (but recommended)
Use in joins	Used as the target of joins	Used as the source of joins

REAL-WORLD EXAMPLE

sql

```
-- Location table (Parent)
CREATE TABLE Location (
    loc_id INT PRIMARY KEY,
    loc_name VARCHAR(50)
```

```

);

-- Department table (Child of Location)
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50),
    loc_id INT, -- Foreign key referencing Location
    FOREIGN KEY (loc_id) REFERENCES Location(loc_id)
);

-- Employee table (Child of Department)
CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(100),
    dept_id INT, -- Foreign key referencing Department
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

```

RELATIONSHIP CHAIN:

text

```

Location (parent)
    ↓ (Foreign Key)
Department (child of Location, parent of Employee)
    ↓ (Foreign Key)
Employee (child of Department)

```

This ensures:

- Every department exists at a valid location
- Every employee belongs to a valid department
- No orphan records exist in the database